Datum Projection, 2D Mode, and Map Frame

A Deep Dive into How Your EKF Decides
"Where Am I on Earth?" and "What Is My Map Origin?"

Reference Guide for the Multi-Sensor Fusion Project

May 2026

Contents

1 The Big Picture: What Problem Are We Solving?

Your robot lives in a Gazebo simulation. It has a GPS sensor that reports latitude and longitude (e.g., 49.9°N, 8.9°E). But your EKF works in *meters*—it tracks position as (x, y) on a flat 2D plane.

So the fundamental questions are:

1. **How do we go from lat/lon (curved Earth) to meters (flat plane)?** → This is the *datum projection* problem.
2. **Where is $(0,0)$ on that flat plane?** → This is the *datum origin* (map origin) problem.
3. **Which direction is "forward" $(+X)$?** → This is the *heading alignment* problem.
4. **Why do we only track x, y, ψ and not z, roll, pitch?** → This is the *2D mode* choice.

This document walks through each of these, showing exactly how your project's configuration files answer each question.

2 Step 1: The Gazebo World Defines "Where on Earth"

2.1 Spherical Coordinates in `agriculture.world`

The Gazebo world file contains this block:

```
<spherical_coordinates>
  <surface_model>EARTH_WGS84</surface_model>
  <world_frame_orientation>ENU</world_frame_orientation>
  <latitude_deg>49.9</latitude_deg>
  <longitude_deg>8.9</longitude_deg>
  <elevation>0</elevation>
  <heading_deg>0</heading_deg>
</spherical_coordinates>
```

What each line means:

`EARTH_WGS84` — The Earth is modeled as the WGS-84 ellipsoid, the same mathematical shape that every GPS satellite uses. It's not a perfect sphere—it's slightly squished at the poles (equatorial radius = 6378.137 km, polar radius = 6356.752 km).

`ENU` — "East–North–Up." This tells Gazebo how to orient its X, Y, Z axes relative to the Earth:

- $+X$ = East
- $+Y$ = North
- $+Z$ = Up

`latitude=49.9, longitude=8.9` — The Gazebo world origin $(0,0,0)$ corresponds to this point on Earth. It's near Darmstadt, Germany. When the robot is at Gazebo position $(0,0,0)$, the GPS sensor will report approximately 49.9°N, 8.9°E.

`heading=0` — No rotation between the ENU geographic frame and the Gazebo world frame. Gazebo $+X$ *is* geographic East.

Key Idea

The spherical coordinates block is what makes GPS work in Gazebo. Without it, the NavSat plugin has no way to convert the robot's (x, y, z) position in the simulator back to latitude/longitude. It defines the "anchor point" between the simulated flat world and the curved Earth.

2.2 How the NavSat Sensor Generates GPS Readings

The `gz::sim::systems::NavSat` plugin does this every 0.1 s (10 Hz):

1. Read the robot's current Gazebo pose: (x_{gz}, y_{gz}, z_{gz}) in meters.
2. Use the WGS-84 inverse projection to convert this to (φ, λ, h):

$$\varphi = \varphi_0 + \frac{y_{gz}}{R_{meridian}} \cdot \frac{180}{\pi} \tag{1}$$

$$\lambda = \lambda_0 + \frac{x_{gz}}{R_{prime} \cos \varphi_0} \cdot \frac{180}{\pi} \tag{2}$$

   where $R_{meridian} \approx 6.366 \times 10^6$ m and $R_{prime} \approx 6.389 \times 10^6$ m at 49.9°N.
3. Add Gaussian noise: $\sigma_{horiz} = 4.5 \times 10^{-6}$ deg ≈ 0.5 m.
4. Publish as `sensor_msgs/NavSatFix` on `/gps/fix`.

Why This Matters

The noise is specified in *degrees*, not meters! At this latitude, 1° latitude ≈ 111 km. So 4.5×10^{-6} degrees ≈ 0.5 m. This is a common source of confusion when configuring Gazebo GPS sensors.

3 Step 2: From Lat/Lon to Local XY Meters

3.1 The UTM Projection

GPS gives you latitude and longitude, but your EKF thinks in meters. The `navsat_transform_node` bridges this gap using the **UTM (Universal Transverse Mercator)** projection.

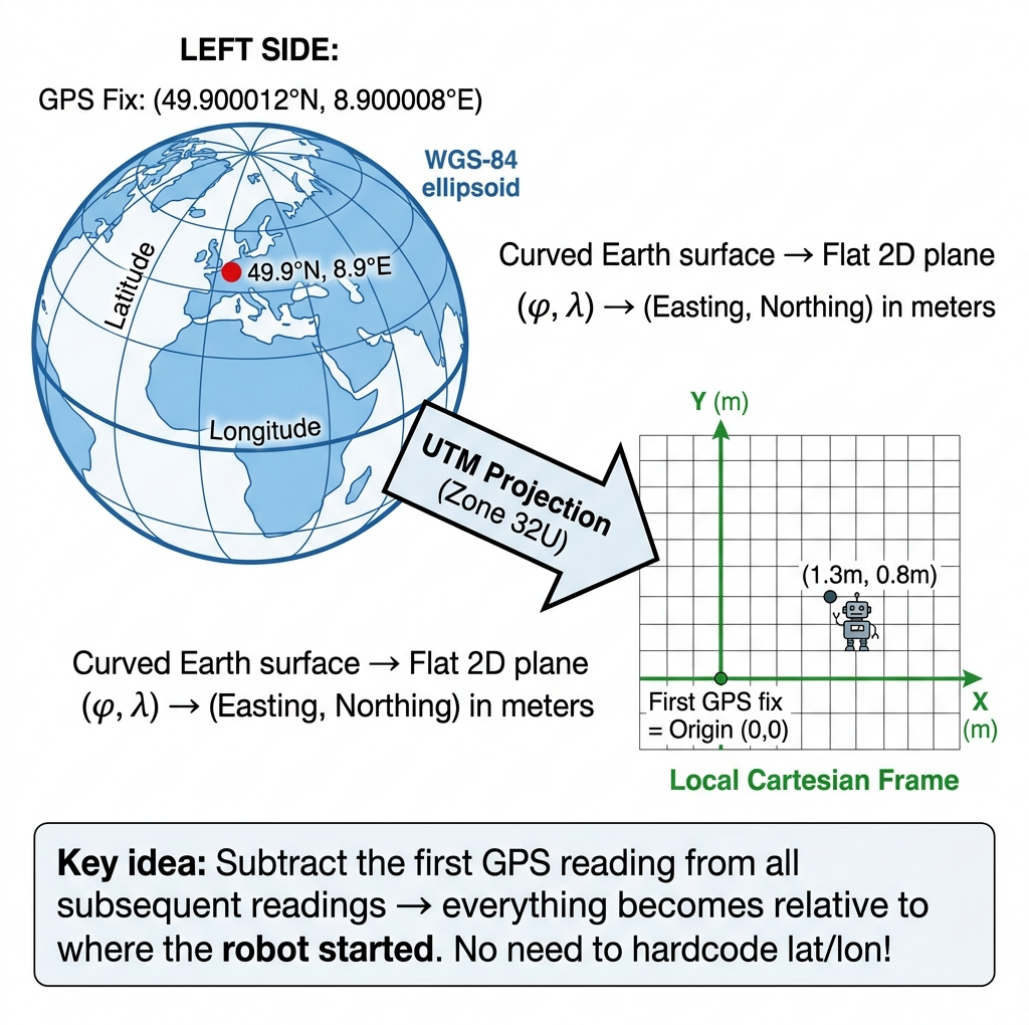

Figure 1: How GPS lat/lon becomes local (x, y) in meters. The UTM projection flattens the curved Earth surface, and subtracting the first GPS fix makes everything relative to the robot's starting position.

UTM divides the Earth into 60 vertical strips ("zones"), each 6° wide. For your world origin at 8.9°E, this falls in **Zone 32U** (covering 6°E to 12°E). Within each zone, UTM uses a transverse Mercator projection to convert (φ, λ) to (E, N) in meters:

- E = Easting (meters east of the zone's central meridian, offset by 500 km)
- N = Northing (meters north of the equator)

The key property: over your robot's ∼40 m operating range, the UTM projection introduces $< 0.04\%$ distortion. A 40 m drive in reality becomes 39.984 m in UTM—a 0.016 m error that's completely negligible compared to your 0.5 m GPS noise.

3.2 The First-Fix Datum: Where Is (0, 0)?

Your Configuration

From `baseline_ekf.yaml`, navsat section:

```
wait_for_datum: false
```

This single parameter determines *how the map origin is chosen*:

- `wait_for_datum: true` would mean: "I will tell you the exact lat/lon to use as $(0,0)$." You'd provide a `datum` parameter with hardcoded coordinates.
- `wait_for_datum: false` (your setting) means: "Use the **first GPS fix you receive** as the origin."

In your system, the first GPS fix arrives at $t \approx 18$ s (after Gazebo loads, sensors start, and the staged launch gates pass). At that moment, the robot is still at its spawn position $(0,0,0)$ in Gazebo. So:

$$\text{First GPS fix:} \quad (\varphi_{\text{first}}, \lambda_{\text{first}}) \approx (49.9^\circ\text{N},\ 8.9^\circ\text{E}) \tag{3}$$

$$\text{UTM conversion:} \quad (E_{\text{first}}, N_{\text{first}}) = (488\,xxx\,\text{m},\ 5\,527\,xxx\,\text{m}) \tag{4}$$

$$\text{Local origin:} \quad (0,0) \leftarrow \text{this position} \tag{5}$$

Every subsequent GPS fix is then computed as:

$$(x,y)_{\text{local}} = (E - E_{\text{first}},\ \ N - N_{\text{first}}) \tag{6}$$

Key Idea

The map origin is wherever the robot happened to be when the navsat node received its first GPS fix. No hardcoded coordinates needed. This is why `wait_for_datum: false` is the more portable choice—your system works in any Gazebo world without changing config files.

4 Step 3: Heading Alignment—Which Way Is "Forward"?

Converting lat/lon to meters gives you (x,y) positions, but there's still an ambiguity: which direction does the local $+X$ axis point? The navsat node needs a heading reference to rotate the UTM frame into the robot's local frame.

Your Configuration

```
use_odometry_yaw: true
magnetic_declination_radians: 0.0
yaw_offset: 0.0
```

`use_odometry_yaw: true` is the critical setting. It means:

> "Take the heading (yaw) from `/odometry/local` (EKF #1's output), not from the IMU's absolute orientation."

4.1 Why Not Use IMU Yaw Directly?

In the real world with a magnetometer, the IMU knows which way is North. But your Gazebo IMU has **no magnetometer**. Its "absolute" yaw is just the integrated gyroscope—which starts at some arbitrary value and drifts over time ($> 150^\circ$ drift in 2 minutes!).

Worse, there's a subtler problem: even if the IMU *did* report true North, the angle between Gazebo world-X and geographic East at 49.9°N is $\sim 58^\circ$. Using IMU yaw would rotate your entire map frame by 58°, misaligning everything.

4.2 How `use_odometry_yaw: true` Solves This

EKF #1 starts with $\psi = 0$ (the robot spawns at heading zero). At $t = 18$ s when navsat initializes, EKF #1's heading is still $\psi \approx 0$. The navsat node reads this and says: "The robot's forward direction aligns with my local $+X$ axis." This means:

- `map` frame $+X$ = same direction as `odom` frame $+X$ at startup
- No mysterious rotation between frames
- `map`→`odom` starts as identity (zero translation, zero rotation)

5 Step 4: The Three Coordinate Frames

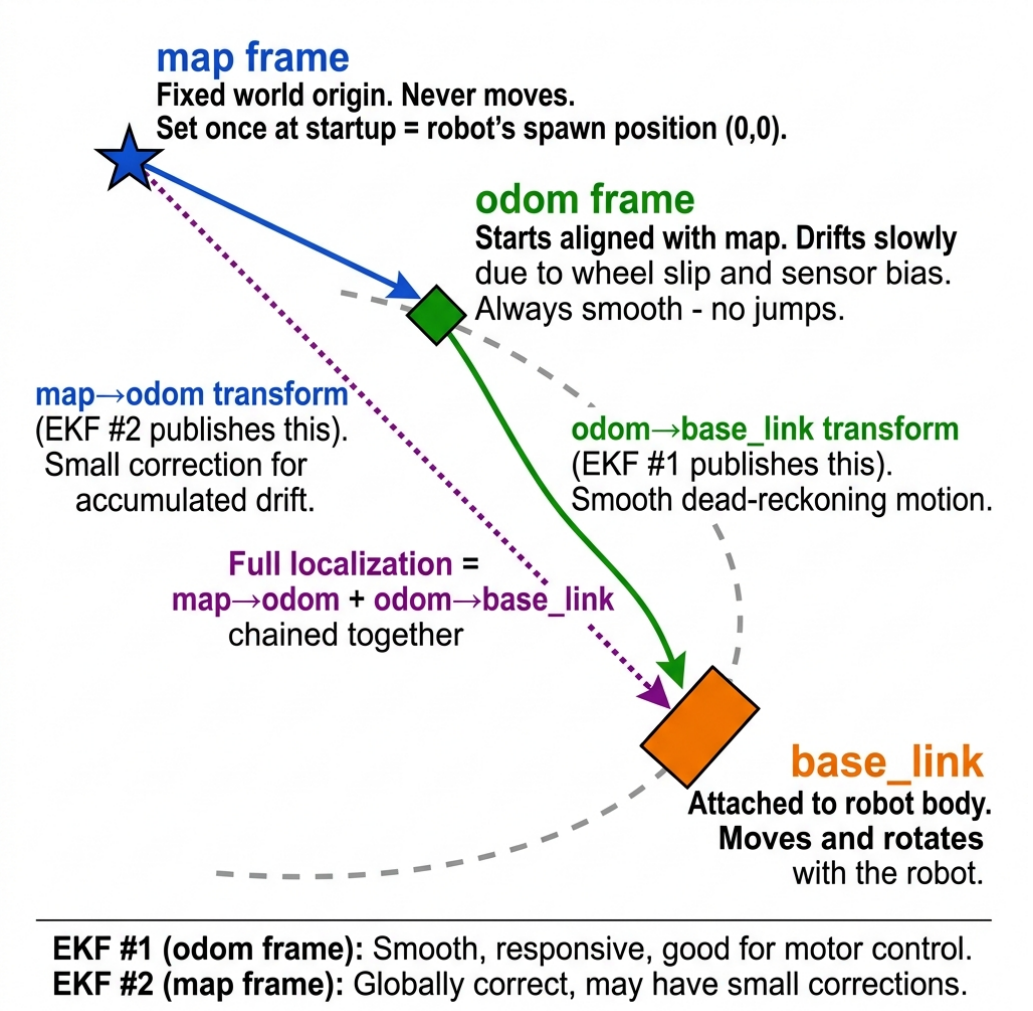

Figure 2: The three coordinate frames in the ROS 2 TF tree. The `map` frame is fixed, `odom` drifts smoothly, and `base_link` moves with the robot. Two EKFs each publish one transform in this chain.

5.1 Frame Definitions

`map` **The global fixed frame.** Origin = robot's spawn position. Never moves. This is where "the world" is. If you ask "where is the robot in the world?" you want the robot's pose in the `map` frame.

`odom` **The smooth local frame.** Starts at the same place as `map`. Over time, it *drifts* because wheel odometry accumulates small errors (wheel slip, uneven terrain). But it's **always smooth**—no sudden jumps. This makes it ideal for motion control (sending velocity commands).

`base_link` **The robot body frame.** Attached to the robot's center. Moves and rotates with the robot. All sensor data is ultimately referenced to this frame.

5.2 Who Publishes What

Transform	**Published by**	`world_frame`	**Meaning**
`odom`→`base_link`	EKF #1	`odom`	Smooth local motion
`map`→`odom`	EKF #2	`map`	Drift correction

Table 1: Transform responsibilities in the dual-EKF architecture.

The full robot pose in the world is the *chain*:

$$T_{\texttt{map}}^{\texttt{base\_link}} = T_{\texttt{map}}^{\texttt{odom}} \cdot T_{\texttt{odom}}^{\texttt{base\_link}} \tag{7}$$

Key Idea

The `world_frame` parameter in each EKF's YAML determines which transform that EKF publishes:

- `world_frame: odom` ⇒ publishes `odom`→`base_link`
- `world_frame: map` ⇒ publishes `map`→`odom`

That's it. That single parameter controls which frame the EKF "owns."

6 Step 5: 2D Mode—Projecting to the Ground Plane

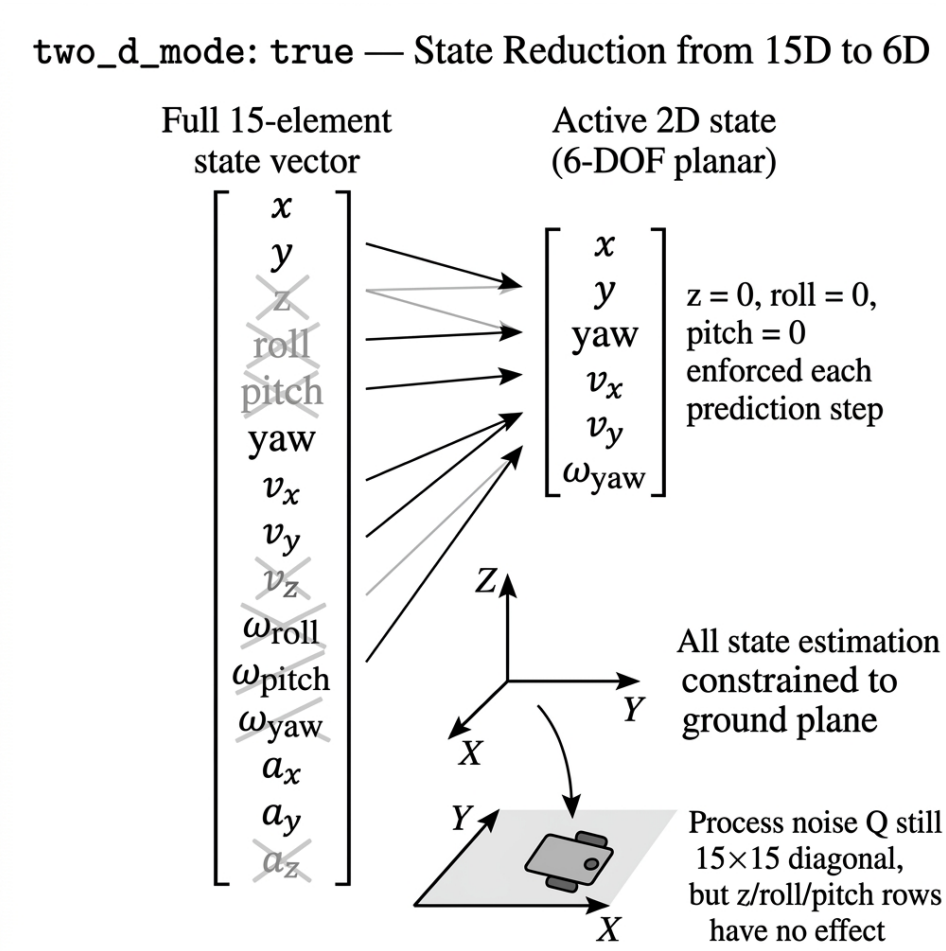

Figure 3: The `two_d_mode: true` setting reduces the 15-element state vector to an effective 6-DOF planar state. Greyed/crossed elements are zeroed at each prediction step.

6.1 The Full 15-Element State

The `robot_localization` EKF internally maintains a 15-element state:

$$\mathbf{x} = [\underbrace{x, y, z}_{\text{position}}, \underbrace{\phi, \theta, \psi}_{\text{orientation}}, \underbrace{\dot{x}, \dot{y}, \dot{z}}_{\text{lin. vel.}}, \underbrace{\dot{\phi}, \dot{\theta}, \dot{\psi}}_{\text{ang. vel.}}, \underbrace{\ddot{x}, \ddot{y}, \ddot{z}}_{\text{lin. accel.}}]^\top \tag{8}$$

For a ground robot driving on a roughly flat surface, we don't care about:

- z (height)—the robot stays on the ground
- ϕ (roll) and θ (pitch)—the robot doesn't tip over
- $\dot{z}, \dot{\phi}, \dot{\theta}, \ddot{z}$—vertical dynamics

6.2 What `two_d_mode: true` Actually Does

At **every predict step** (50 times per second), the EKF:

1. Runs the normal prediction: $\hat{\mathbf{x}}_k^- = f(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k)$

2. Then **forces**: $z = 0$, $\phi = 0$, $\theta = 0$

3. Also zeros the corresponding velocities: $\dot{z} = 0$, $\dot{\phi} = 0$, $\dot{\theta} = 0$

The effective state becomes 6-DOF planar:

$$\mathbf{x}_{2D} = [x,\ y,\ \psi,\ \dot{x},\ \dot{y},\ \dot{\psi}]^\top \tag{9}$$

Why This Matters

The process noise matrix $\mathbf{Q}$ is still specified as 15×15 in the YAML. The entries for z, roll, pitch rows still exist—they just don't accumulate because the state is zeroed. You can set them to anything; they have no effect in 2D mode. But the entries for x, y, ψ, v_x, v_y, ω_z **do matter** and must be tuned carefully.

7 Step 6: Why GPS Is Excluded from EKF #2

This is the most counter-intuitive design decision in the system. The standard dual-EKF pattern from the `robot_localization` tutorials puts GPS into EKF #2 to "anchor" the map frame globally. Your system deliberately does **not** do this. Here's why.

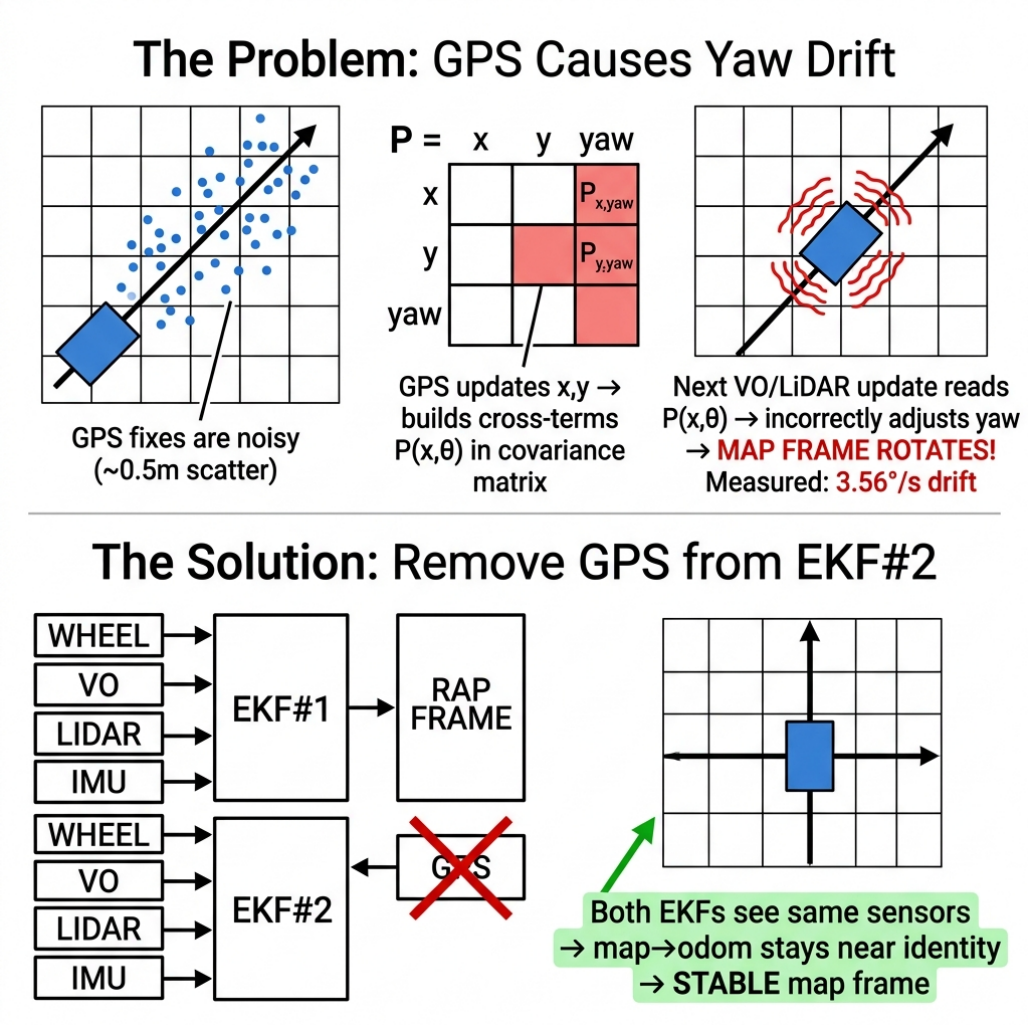

Figure 4: The GPS position–yaw covariance coupling problem. GPS corrections build off-diagonal $\mathbf{P}_{x\theta}$ terms that leak into yaw when VO or LiDAR updates are applied, causing sustained map-frame rotation at $\sim$3.56°/s.

7.1 The Covariance Cross-Term Problem

The EKF's state covariance matrix $\mathbf{P}$ has off-diagonal terms. When GPS updates the position (x, y), the Kalman gain computation involves *all* states, including yaw ψ. Over multiple GPS corrections, the off-diagonal terms $P_{x,\psi}$ and $P_{y,\psi}$ grow. Then when a visual or LiDAR odometry update arrives and corrects the position, the cross-terms cause the EKF to *also* adjust yaw—even though the VO/LiDAR update had nothing to do with heading.

The result: the `map` frame slowly rotates. Measured drift: $\approx 3.56°/s$, which is catastrophic for any downstream navigation.

7.2 The Solution: Sensor Consensus

By giving EKF #2 the *same* sensor inputs as EKF #1 (just in the `map` world frame), both filters track nearly the same state. The `map`→`odom` transform stays near-identity: ± 0.1 m translation, $\pm 1°$ rotation over a 2-minute run.

The GPS data still flows through the system (navsat converts it to `/odometry/gps`), but that topic is simply not listed in EKF #2's sensor inputs. It's available for analysis notebooks but doesn't corrupt the map frame.

8 Step 7: The Complete Data Flow

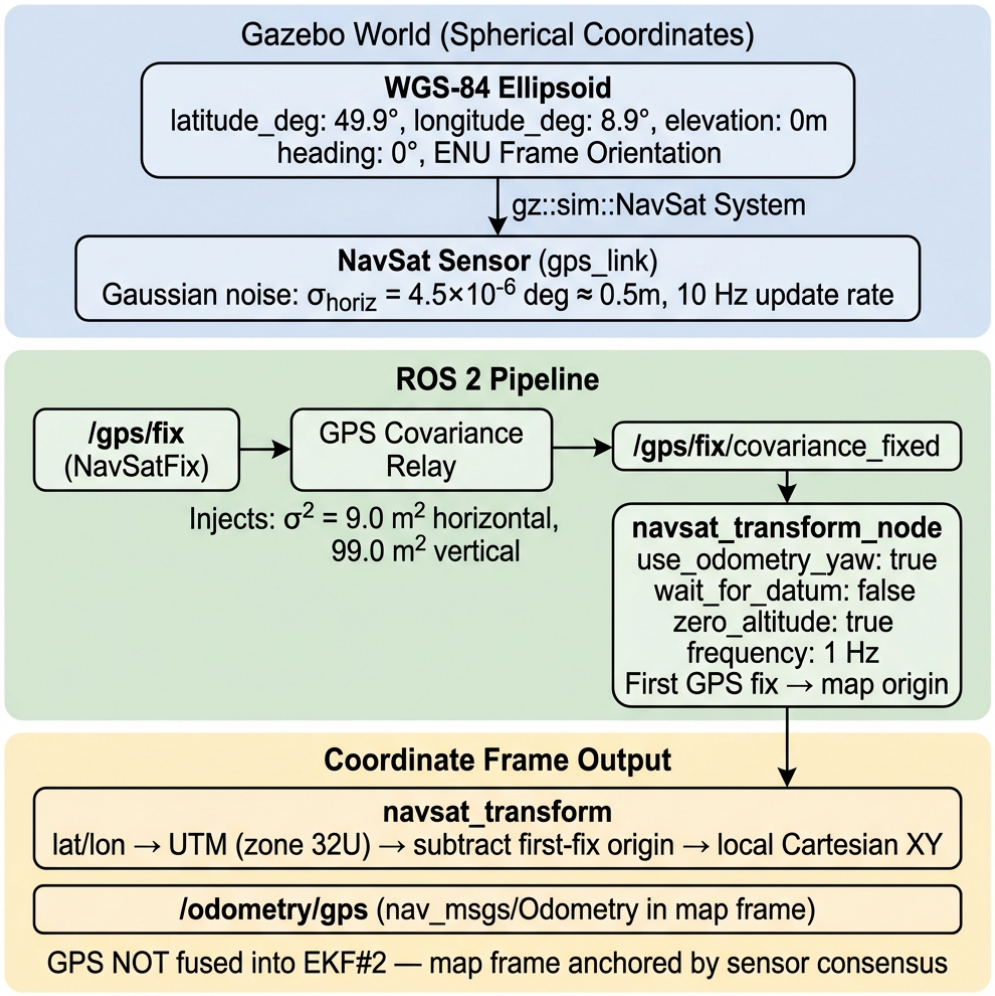

Figure 5: The complete GPS datum projection pipeline from Gazebo world coordinates through the ROS 2 processing chain.

Putting it all together, the data flows like this:

1. **Gazebo physics** computes robot pose (x, y, z) in meters.
2. **NavSat plugin** converts to (φ, λ) using WGS-84, adds noise ($\sigma = 0.5$ m), publishes `/gps/fix`.
3. **GPS Covariance Relay** injects realistic covariance ($\sigma^2 = 9.0\,\text{m}^2$), publishes `/gps/fix/covariance_fixed`.
4. **navsat_transform** receives first fix → locks datum origin. Reads EKF#1 heading ($\psi = 0$) → aligns axes. All future fixes: UTM project, subtract origin → `/odometry/gps` in local meters.
5. **EKF #1** fuses wheel + VO + LiDAR + IMU in `odom` frame → publishes `odom`→`base_link`.
6. **EKF #2** fuses same sensors in `map` frame (NO GPS!) → publishes `map`→`odom`.

9 Step 8: The TF Frame Hierarchy

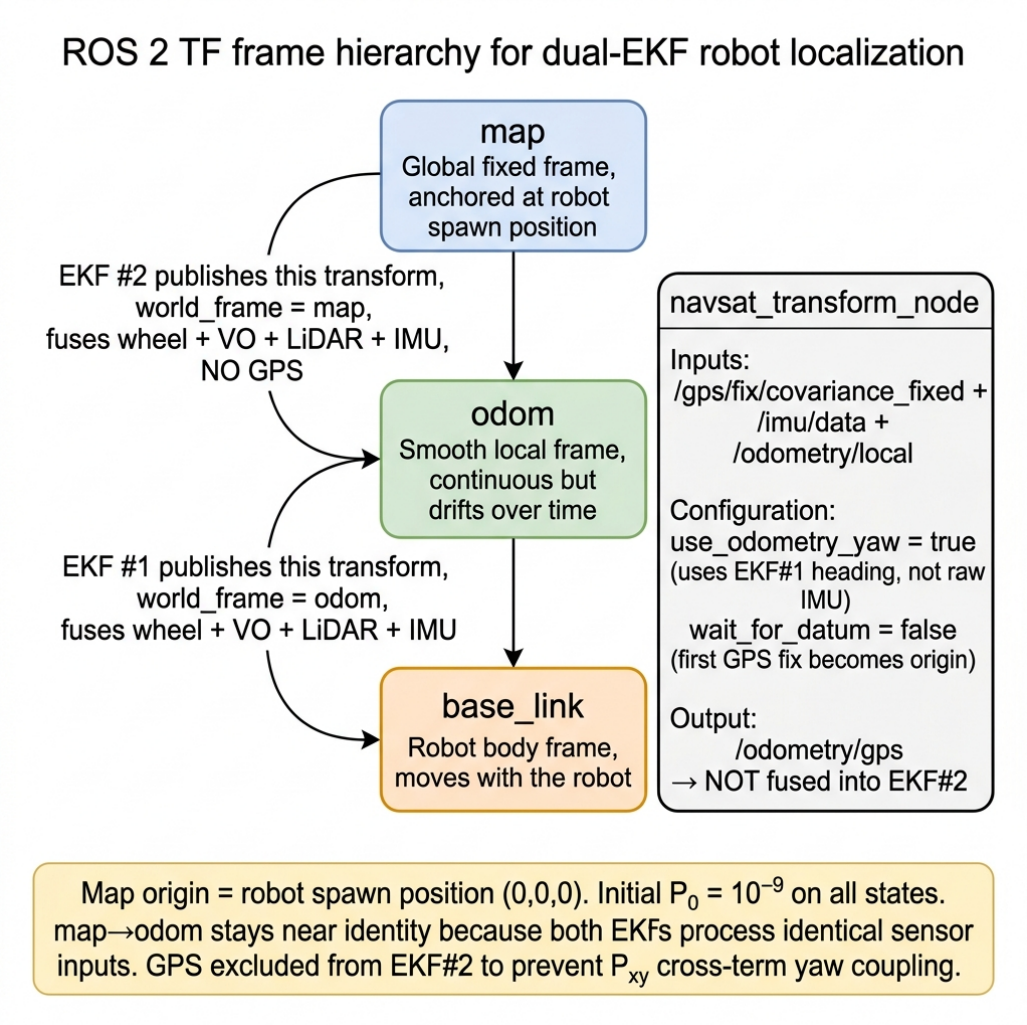

Figure 6: The ROS 2 TF frame hierarchy showing which node publishes each transform, and how the navsat_transform node feeds into (but is excluded from) EKF #2.

9.1 Initial Covariance: The 10^{-9} Lock

Both EKFs set their initial state covariance to:

$$\mathbf{P}_0 = 10^{-9} \cdot \mathbf{I}_{15} \tag{10}$$

This is *extremely* small—it tells the filter: "I am almost perfectly certain the robot is at $(0,0,0)$ with heading 0." Why?

Because the robot *is* at $(0,0,0)$—it just spawned there. With such tight initial covariance:

- The Kalman gain for any early measurement is $K \approx 10^{-10}$
- No single sensor can yank the state away from the known starting point
- Covariance grows gradually via process noise $\mathbf{Q}$, so sensors gain influence slowly and smoothly

10 RTAB-Map Visual Odometry (RGB-D)

The `rgbd_odometry` node from the `rtabmap_odom` package estimates the robot's motion by tracking visual features across consecutive camera frames. Fig. 7 shows the complete pipeline.

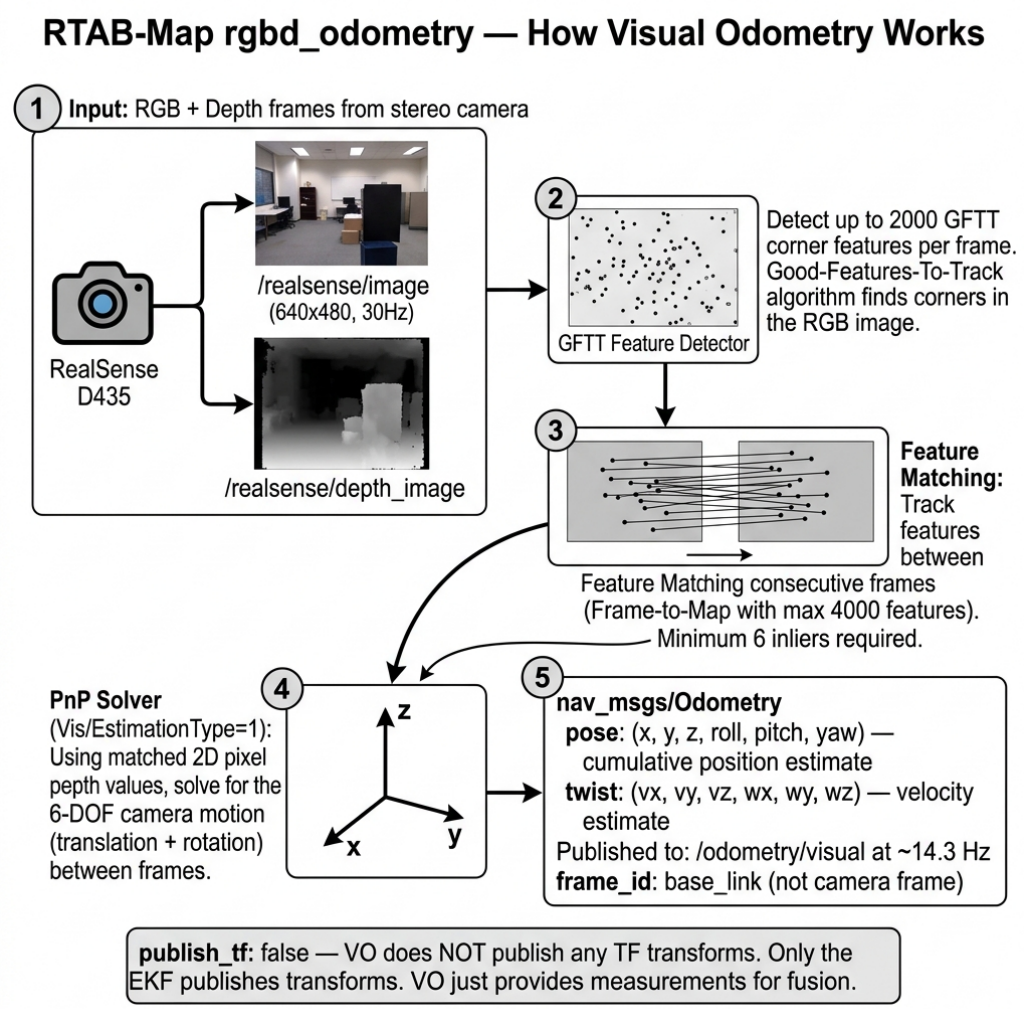

Figure 7: RTAB-Map RGB-D visual odometry pipeline. The RealSense D435 provides synchronized color and depth images. GFTT features are detected, matched across frames, and a PnP solver recovers the 6-DOF camera motion.

10.1 Inputs

- **RGB image:** `/realsense/image` — 640×480 color frame at 30 Hz from the Intel RealSense D435.
- **Depth image:** `/realsense/depth_image` — aligned depth map, same resolution. Each pixel stores distance in meters (range: 0.105–10 m).
- **Camera info:** `/realsense/camera_info` — intrinsic calibration (f_x, f_y, c_x, c_y, distortion). Needed to back-project 2D pixels into 3D points.

10.2 How It Works (Step by Step)

1. **Feature detection (GFTT):** The Good-Features-To-Track algorithm (Shi-Tomasi corners) finds up to 2000 corner points in the RGB image. Corners are places where intensity changes in two

directions—intersections, edges of objects, texture boundaries. Your config uses `Vis/FeatureType: 0` = GFTT, and `Vis/MaxFeatures: 2000` (raised from default 1000 because the agriculture terrain is visually sparse).

2. **Feature matching (Frame-to-Map):** Detected features are matched against a local map of previously seen features (up to 4000 features in the map via `OdomF2M/MaxSize: 4000`). Matching uses descriptor similarity. At least 6 inlier matches are required (`Vis/MinInliers: 6`)—if fewer are found, the frame is rejected and the odometry resets after 5 consecutive failures (`Odom/ResetCountdown: 5`).

3. **3D point recovery:** For each matched 2D feature, the corresponding depth pixel provides the Z distance. Combined with the camera intrinsics, this gives a 3D point (X, Y, Z) in camera coordinates:

$$X = \frac{(u - c_x) \cdot Z}{f_x}, \quad Y = \frac{(v - c_y) \cdot Z}{f_y} \tag{11}$$

4. **PnP solver:** With matched 2D↔3D correspondences, the Perspective-n-Point algorithm (`Vis/EstimationType: 1`) solves for the rigid-body transform (R, t) that best explains the observed feature motion. This gives the 6-DOF camera displacement between frames.

5. **Transform to base_link:** The camera-frame displacement is transformed to `base_link` using the known camera mount transform (the `body_to_camera_joint` in the URDF: $x = 0.38$, $z = 0.20$, pitch $= 0.35$ rad down).

10.3 Output

A `nav_msgs/Odometry` message on `/odometry/visual` at ∼14.3 Hz containing:

- **pose:** Cumulative $(x, y, z, \text{roll}, \text{pitch}, \text{yaw})$ — the total displacement from where VO started.
- **twist:** Instantaneous $(v_x, v_y, v_z, \omega_x, \omega_y, \omega_z)$ — velocity computed from consecutive frame deltas.
- **covariance:** 6×6 matrix auto-computed from the number of inliers and reprojection error.
- **frame_id:** `base_link` (not `camera_link`).

Why This Matters

`publish_tf: false` — The VO node does NOT publish any TF transforms. Only the EKF publishes transforms. VO is purely a measurement source. If this were `true`, you'd have two nodes fighting over the same TF transform and the robot would jitter violently in RViz.

11 RTAB-Map ICP LiDAR Odometry

The `icp_odometry` node estimates motion by aligning consecutive 3D LiDAR scans using the Iterative Closest Point algorithm. Fig. 8 shows the pipeline.

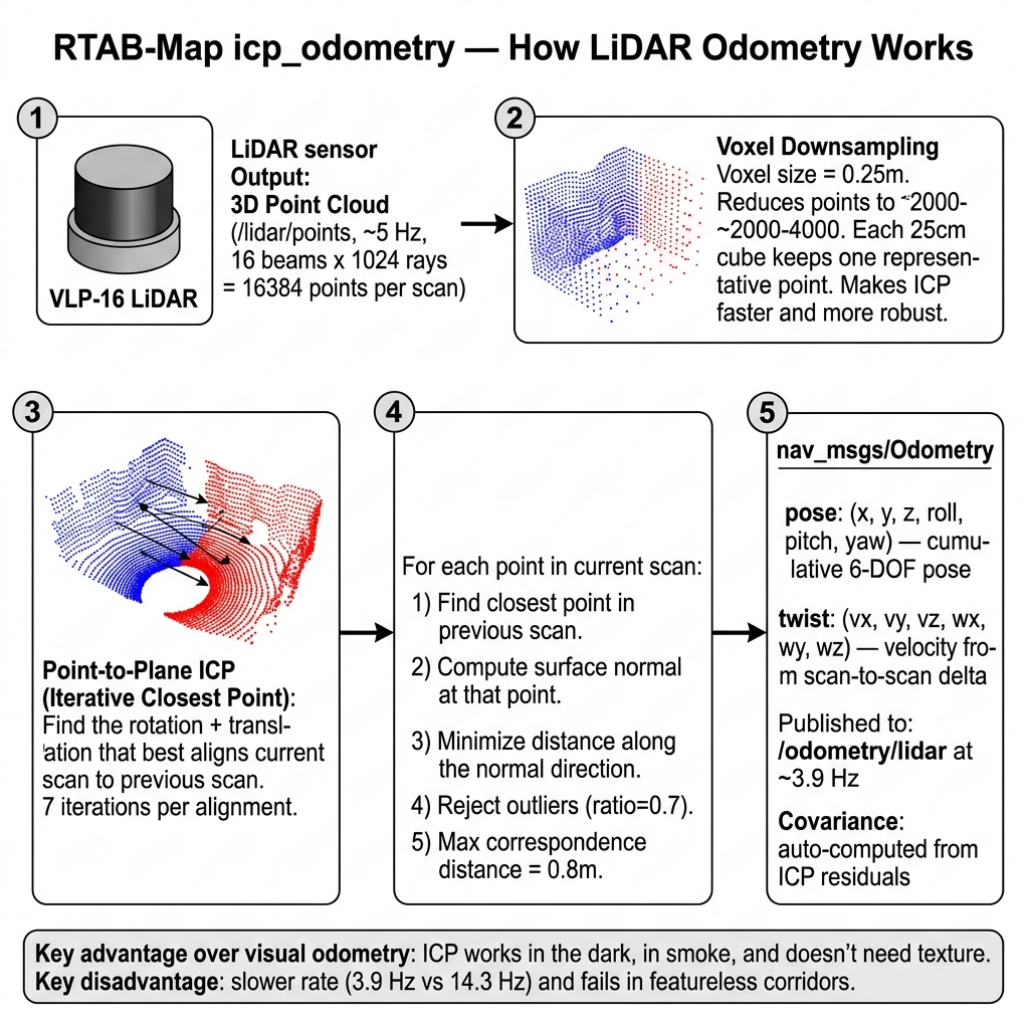

Figure 8: RTAB-Map ICP LiDAR odometry pipeline. Point clouds from the VLP-16 are voxel-downsampled, then aligned to the previous scan using point-to-plane ICP to recover 6-DOF motion.

11.1 Inputs

- **Point cloud:** `/lidar/points` — `PointCloud2` from the VLP-16 at ∼5 Hz. 16 vertical beams × 1024 horizontal rays = up to 16384 points per scan, covering 360° horizontal, ±15° vertical.

11.2 How It Works (Step by Step)

1. **Voxel downsampling:** The raw cloud (∼16K points) is divided into $0.25 \times 0.25 \times 0.25$ m voxels (`Icp/VoxelSize: 0.25`). Each voxel keeps one representative point (centroid). Result: ∼2000–4000 points. This makes ICP faster and more robust to noise.

2. **Point-to-Plane ICP:** (`Icp/PointToPlane: true`) — For each point $\mathbf{p}_i$ in the current scan, find the closest point $\mathbf{q}_i$ in the previous scan. Compute the surface normal $\hat{\mathbf{n}}_i$ at $\mathbf{q}_i$. Then minimize:

$$\min_{R,t} \sum_i \left[(R\,\mathbf{p}_i + t - \mathbf{q}_i) \cdot \hat{\mathbf{n}}_i \right]^2 \tag{12}$$

   This is iterated 7 times (`Icp/Iterations: 7`). Point-to-plane converges faster than point-to-point because it allows points to "slide" along surfaces.

3. **Outlier rejection:** Points whose correspondence distance exceeds 0.8 m (`Icp/MaxCorrespondenceDistance`) are ignored. Additionally, 30% of the worst-matching points are discarded (`Icp/OutlierRatio: 0.7` means keep 70%).

4. **Convergence check:** If the transform change between iterations drops below $\varepsilon = 0.001$ (`Icp/Epsilon`), ICP stops early. Max translation per step is clamped to 1.0 m (`Icp/MaxTranslation`).

5. **Keyframe management:** A new keyframe is created when the cumulative motion exceeds 50% of the scan overlap (`Odom/ScanKeyFrameThr: 0.5`). Between keyframes, ICP aligns against the stored keyframe scan.

11.3 Output

A `nav_msgs/Odometry` message on `/odometry/lidar` at $\sim$3.9 Hz containing the same fields as visual odometry (pose, twist, covariance), but with covariance auto-computed from the ICP residual error.

11.4 Visual Odom vs. LiDAR Odom: Key Differences

Property	**Visual (RGB-D)**	**LiDAR (ICP)**
Sensor	RealSense D435	VLP-16
Rate	$\sim$14.3 Hz	$\sim$3.9 Hz
Range	0.1–10 m	0.4–100 m
FOV	$87^\circ \times 57^\circ$	$360^\circ \times 30^\circ$
Failure mode	Featureless surfaces	Long featureless corridors
Yaw accuracy	Poor (agri terrain)	Good (360° scan)
EKF uses yaw?	**No**	**Yes**

Table 2: Comparison of the two RTAB-Map odometry sources.

12 How the EKF Consumes Odometry: Differential Mode

Both VO and LiDAR odom are consumed with `differential: true`. This is a crucial setting that determines *how* the EKF interprets the pose data (Fig. 9).

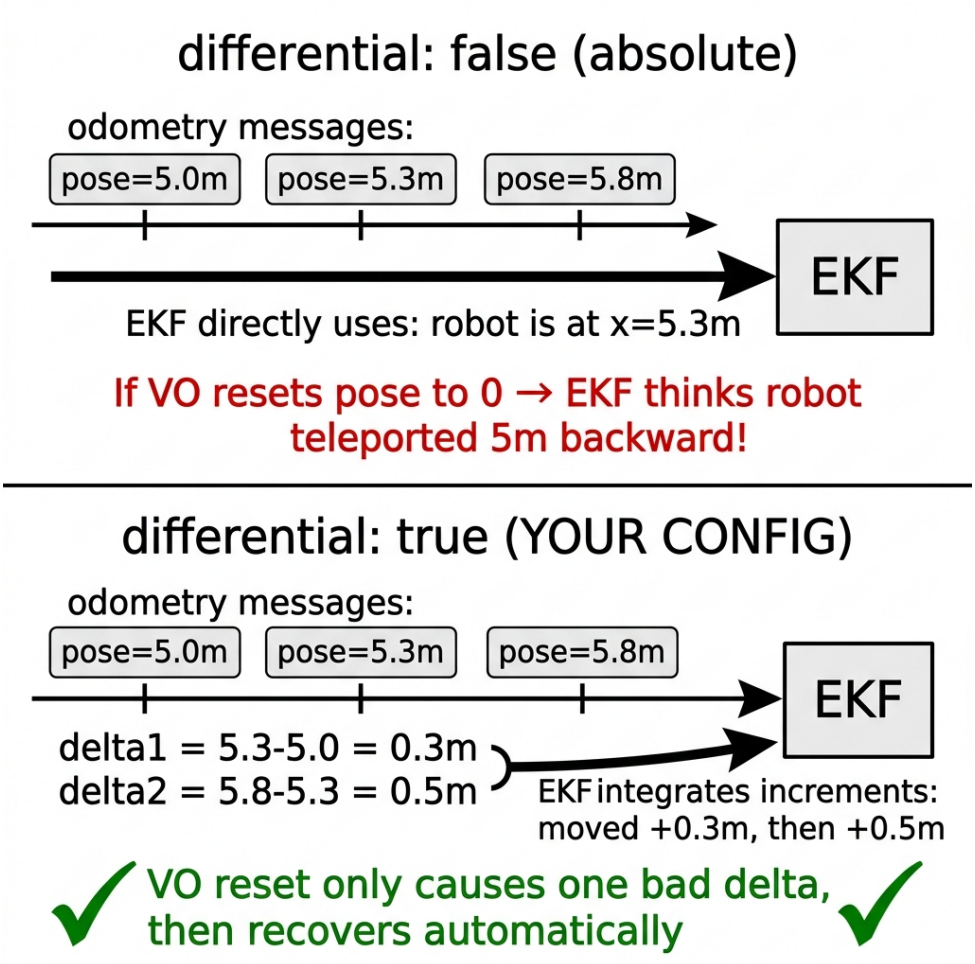

Figure 9: Differential mode converts absolute poses to increments before EKF fusion. This protects against VO/LiDAR resets corrupting the state estimate.

Key Idea

With `differential: true`, the EKF never sees "robot is at position X"—it only sees "robot moved ΔX since the last message." This means RTAB-Map can internally reset its pose estimate (after tracking failures) without destroying the EKF state. Only one update will be wrong (the delta spanning the reset), and the system self-recovers on the next good frame.

13 The Sensor Selection Matrix

The 15-element boolean `config` array for each sensor determines exactly which fields the EKF reads. Fig. 10 shows the full picture.

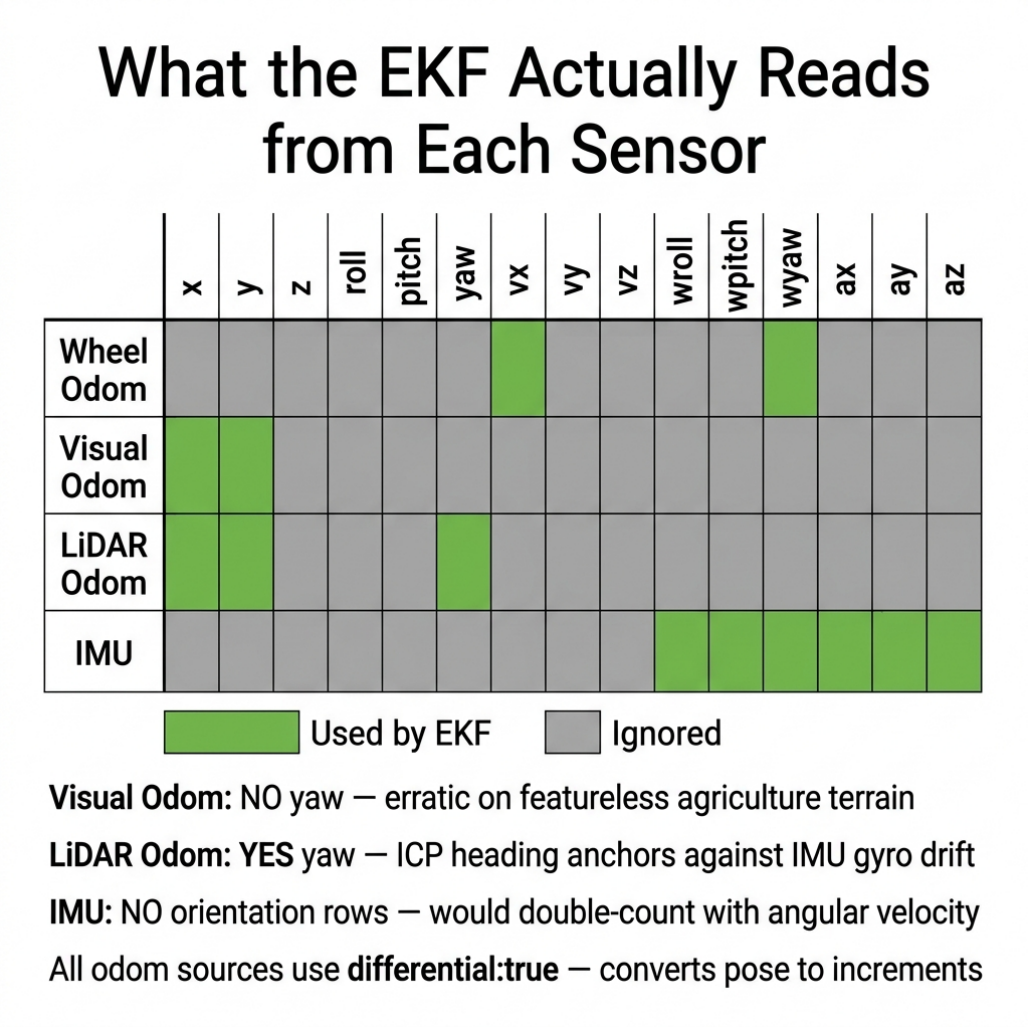

Figure 10: Which state variables each sensor contributes to the EKF. Green = used, grey = ignored. Note: VO provides no yaw (erratic on agriculture terrain), while LiDAR provides yaw (reliable 360° ICP heading).

Key design decisions in the sensor matrix:

- **Wheel odom:** Only v_x and ω_z (velocities). Position is not used because wheel odometry position accumulates unbounded drift from wheel slip.
- **Visual odom:** Only x, y (position, differential). **No yaw** — on the featureless agriculture terrain, VO heading estimates oscillate by 50°+ and corrupt the EKF.
- **LiDAR odom:** x, y **and yaw** (differential). ICP on 360° scans gives reliable heading that anchors against IMU gyro bias drift (∼3°/min without it).
- **IMU:** Angular velocity $(\omega_x, \omega_y, \omega_z)$ and linear acceleration (a_x, a_y, a_z). **No orientation rows** — fusing both orientation and angular velocity would double-count the same gyroscope signal, amplifying bias.

14 Summary: Your Config Decisions at a Glance

Parameter	Your Value	Effect
`spherical_coordinates`	49.9°N, 8.9°E	Gazebo world origin on WGS-84
`world_frame_orientation`	ENU	$+X$=East, $+Y$=North, $+Z$=Up
NavSat noise σ	4.5×10^{-6} deg	≈ 0.5 m GPS scatter
`wait_for_datum`	`false`	First GPS fix = map origin
`use_odometry_yaw`	`true`	Heading from EKF #1 (not IMU)
`zero_altitude`	`true`	Forces $z = 0$ in GPS output
`two_d_mode`	`true`	15D $\rightarrow$ 6D state
EKF #1 `world_frame`	`odom`	Publishes odom$\rightarrow$base_link
EKF #2 `world_frame`	`map`	Publishes map$\rightarrow$odom
GPS in EKF #2	**excluded**	Prevents $P_{x\theta}$ yaw coupling
$\mathbf{P}_0$ diagonal	10^{-9}	Locks to known spawn position
NavSat frequency	1 Hz	Slow GPS $\rightarrow$ less map jitter

Table 3: All datum and frame-related configuration decisions.